Real-Drone Day Plan

Team 13 - Stanford DroneHacks 2026 - Saturday, November 7, 2026

What the on-site team does tomorrow, in order: what to check, what to measure, what to record, how to fly in stages, and what to send back. Prepared from the work of Nov 6, 2026.

1. Where we are tonight

Item	Status (Nov 6, late)
Flight stack	One Python/ROS 2 stack in `cartographer/official/`: mission state machine (full course), depth obstacle map, floor / tunnel / window / white-rock detectors, PX4 offboard node, flight recorder. All unit tests pass.
Simulation	Ubuntu Gazebo replica of the venue, built from 7 photos, with interfaces matching the official lab. With ideal rendered depth, the full course was flown 2/2 with unchanged code (~57 s, landing 0.1-3 cm from start).
Realistic depth	With OAK-like stereo (SGBM) depth: 0/4 before tonight's fixes. Phantom obstacles caused side-steps into unseen panels. Fixes landed: phantom carving in the map, small-cluster rule, no blind side-steps, tunnel/window interiors cleared after lock, white-rock latch. Re-flights with randomly offset priors are running tonight.
Not yet verified	Real OAK-D data, real venue dimensions and layout, the official lab sim (GPU queue full), the real PX4 parameters, cuVSLAM on our flight path.

Priority for the day: get real sensor data and real venue numbers first. A hand-carried recording through the course plus a tape-measured layout is worth more than any early flight. Fly only after the interface check and the offline replay pass.

2. Safety and rules (every flight)

- Before any autonomous flight, test the RC takeover / kill switch: one pilot owns it, hand on the sticks.
- Get team approval before every arm. Clear the cage of people. Fly inside the net only.
- First flights low (0.8 m) and slow (v_cruise 0.5 m/s). Raise speed only after a stage passes twice.
- Abort = RC takeover, or the Land command. Stopping the program makes the node command LAND and wait for touchdown.
- No GPS, no CAD, no external positioning. Coarse tape-measured stage positions (+/- 0.5 m) are allowed.
- Keep a fresh battery for each test block. Watch for estimator warnings (/estimator/healthy must stay true).

3. Order of work

#	Step	Output	Go / no-go
0	Power on, props OFF. Install our code: copy `cartographer/official/` to the companion (e.g. `~/claude_dev/`).	Code on the drone	Imports work: `python3 -c "import node.obstacle_map, mission.core"`
1	Interface check (section 4).	Topic list, rates, camera_info, TF, PX4 params saved as files	All required topics present with sane rates
2	Measure the venue (section 5). Write `mission/course_prior.venue.json`.	Prior JSON + sketch + photos	Every field filled

#	Step	Output	Go / no-go
3	Hand-carried walk through the whole course while recording (section 6).	2-3 bags	Bags open, images and depth not black
4	Send the bags and files back. Claude replays VIO, the obstacle map and all detectors offline (~30-60 min).	Replay report	Detectors find tunnel / window / rock; no phantom walls on the path
5	Hover-and-land flight (section 7, stage A).	Recorded flight	Hover steady, PX4 reports landed, disarms
6	Stage flights B-F (section 7).	Recorded flights	Each stage passes twice
7	Full course, slow, then faster.	Recorded flights	Done 2/2 at slow speed

4. Interface check (props off)

Our node expects exactly the official-lab interface. Run these on the companion and save the outputs to files.

Check	Expected	Command / note
Stereo images	`/stereo/left/image`, `/stereo/right/image`: mono8 640x400, ~15 Hz	`ros2 topic hz /stereo/left/image`
Depth	`/camera/depth/image_raw`, aligned to the LEFT camera, 32FC1 m or 16UC1 mm, ~15 Hz	also `/camera/depth/camera_info`
Intrinsics	Left K, roughly fx = fy ~ 421, cx ~ 319, cy ~ 200 (sim values; real calibration wins)	`ros2 topic echo --once /stereo/left/camera_info`
Camera mount	TF base_link to oak_left, horizontal, facing forward; sim value t = (0.116, 0.037, 0.029) m	`ros2 run tf2_ros tf2_echo base_link oak_left`
Down range	`/fmu/out/distance_sensor` valid on the ground and in the air	lift the drone by hand: the range should follow
PX4 topics	`/fmu/out/vehicle_local_position_v1`, `vehicle_status_v1`, `vehicle_attitude`, `vehicle_land_detected`	`ros2 topic list | grep fmu`
Estimator	`/estimator/healthy` true while still and while carried; local position valid	walk the drone 2 m forward: x should grow by about 2 m
PX4 params	Save `LNDMC*`, `MPC*`, `EKF2*`, `COM*`, `NAV*`	`param show` from the PX4 shell, to a file

Landing detection note: in simulation, PX4 1.17 only reports "landed" under offboard while the descent command is at least 1.1 x LNDMC_Z_VEL_MAX. Our descent now holds 0.35 m/s through touchdown, then falls back to PX4 LAND after 4 s of ground contact. Send us LNDMC_Z_VEL_MAX so we can confirm the margin.

5. Venue measurements

Tape-measure everything in the **start frame**: the drone sits on the start pad. **Forward** = the direction its nose points at takeoff, **left** = its left, **up** = above the floor, in metres. Angles are in degrees, counter-clockwise from forward (90 = left, -90 = right, 180 = back). +/- 0.2 m is good; +/- 0.5 m is the limit. Fill the values into `mission/course_prior.venue.json` (same format as `course_prior.example.json`). No code change is needed.

What	How to measure	JSON field
Start pad to each net wall (front, back, left, right); net height (~3.05 m)	Horizontal distance from the pad centre. Back and right are negative.	`fence.forward_max / forward_min / left_max / left_min / up_max` (height minus 0.2)

What	How to measure	JSON field
Nose direction on the pad	Photo of the drone on the pad	Defines "forward": must be the same at every takeoff
Corridor	2-3 points on the corridor centre line (entry, middle, exit), flight height	`corridor[]`: forward / left / up
Tunnel	Centre of the FIRST hoop (incl. height); travel direction; first-to-last hoop length	`tunnel.entry, axis_deg, length_m`
Tunnel (detector)	Hoop radius, hoop spacing, hoop lowest point above the floor	Send to Claude: tunnel detector scale priors
Window	Centre of the opening (incl. height); travel direction through it	`window.center, through_deg`
Window (detector)	Opening width x height; board width x height; sill height	Send to Claude: `known_size_m`
White rock	Centre of the top face; top height; length x width	`landing_target.center, top_up`
Wall	One floor point on the rock side and one on the home side (the line between them must cross the wall); wall top height	`wall.cross_from, cross_to, top_up`

Also take a top-down sketch with every stage, its direction and the real (3-D) rocks marked, plus one photo per stage from the drone's height (~0.8-1.2 m).

If a tape measure is not allowed

Photos plus eyeballing are enough. Aim for +/- 0.5 m and accept up to about 0.8 m. The detectors refine each stage, and tonight's simulation runs test exactly this error level.

- Use the cage as the ruler: the floor is 20 x 20 ft (6.10 m), and the net is 10 ft (3.05 m) high. Estimate distances as fractions of the cage side.
- Pace distances (calibrate one pace against the cage side first). Count floor-print features or panels of known width (the banner panels are about 1.5 m wide in the photos).
- Take overview photos from two cage corners at head height, plus one photo per stage from the start pad. Claude can estimate positions from the perspective and cross-check them against your numbers.
- Directions matter more than distances: note which way each stage is passed (tunnel axis, window travel direction) relative to the drone's nose on the pad.
- Fence numbers: when unsure, be conservative (estimate the net closer than it looks); the drone keeps a margin from these.

6. Recording (hand-carried walk-through)

Carry the drone (props off, armed state NOT needed) along the planned path at about 0.5 m/s, at 0.8-1.2 m height, camera facing the direction of travel. Pause 3 s in front of each stage: tunnel mouth from about 1.8 m, window from about 1.5 m, white rock from about 1.6 m away and 0.5 m above its top. Do it 2-3 times, including one pass with more yaw and height changes. This does not run the mission and cannot arm the drone.

```
ros2 bag record -s mcap --storage-preset-profile zstd_fast -o walk_01 \
  /stereo/left/image /stereo/right/image /stereo/left/camera_info /stereo/right/camera_info \
  /camera/depth/image_raw /camera/depth/camera_info /tf /tf_static \
  /fmu/out/vehicle_local_position_v1 /fmu/out/vehicle_attitude /fmu/out/distance_sensor \
  /estimator/healthy /visual_slam/tracking/odometry /visual_slam/status <OAK IMU topic>
```

Find the IMU topic with `ros2 topic list | grep -i imu`. Expect about 4-6 MB/s with images.

Recorded flights

Every flight goes through the recorder. It creates `flights/<time>_<name>/` with the log, an mcap bag, a code snapshot, the prior and params, the result, and a row in `flights/INDEX.md`. Use **--real** on the aircraft: it switches to the wall clock instead of simulation time.

```
cd ~/claude_dev
python3 tools/flight_run.py --real --name hover_1 --prior mission/course_prior.venue.json \
  --params '{"v_cruise": 0.5, "stop_after": "takeoff", "stop_hover_s": 10}' --notes "pad A, battery
1"
```

Add `--light` to skip camera images in the bag (smaller files). Keep images for the first flights of each stage. Ctrl-C makes the node command LAND, wait for touchdown, then close the bag.

7. Staged flights

Each stage ends with a hold and a landing in place, via `stop_after`. Move to the next stage only after a stage passes twice. Start at `v_cruise` 0.5 m/s.

Stage	--params (JSON)	Pass criteria
A Hover	{"v_cruise": 0.5, "stop_after": "takeoff", "stop_hover_s": 10}	Climbs to 0.8 m, hover drift under 0.15 m, lands, PX4 reports landed, disarms
B Corridor	{"v_cruise": 0.5, "stop_after": "corridor"}	Flies the corridor without side-steps over 0.3 m; no blocked events on free space
C Tunnel detect	{"v_cruise": 0.5, "stop_after": "tunnel_align"}	Tunnel locked (log event latched), aligned point within 0.1 m of the axis; does NOT enter
D Tunnel through	{"v_cruise": 0.5, "stop_after": "tunnel_traverse"}	Passes through, no contact, lands after the exit
E Window	{"v_cruise": 0.5, "stop_after": "window_traverse"}	Window locked with clearance_ok, passes the centre within 0.1 m
F Rock landing	{"v_cruise": 0.5, "stop_after": "land_target"}	Lands on the rock top, ToF matches the rock height, disarms
G Full course	{"v_cruise": 0.8}	Rearm, over the wall, home landing; then raise v_cruise step by step

`stop_after` (with `stop_hover_s`; optional `stop_land_px4`) is implemented and tested. Without it, use a prior whose later stages are far outside the cage: the mission then hovers at the next standoff, times out and lands.

8. Known risks: what to watch

Risk	Why	What to look for
Phantom obstacles from stereo	SGBM mismatches on the regular black net and on dark low-texture rock edges. In sim they caused side-steps into unseen panels.	Log events avoid/blocked on visibly free space; sudden lateral moves. Send the bag.
Low-texture boards / floor	Tan boards and the white rock top give depth holes (60%+ in sim); the glossy floor reflects lights.	Window or rock not latching; floor fit off. The detectors fall back to the priors.
VIO drift	Dark hall, repeated textures, fast yaw. Indoor VIO is the single point of failure.	/estimator/healthy flickers; the position jumps. Walk-through bag first.
Net geofence	Depth barely sees the net; the fence comes only from the measured prior.	A wrong fence number means the drone can touch the net: double-check section 5.
Landing detection	PX4 offboard landing needs a descent command above the land-detector threshold.	PX4 must report "landed" within a few seconds of touchdown.
Real rocks vs painted rocks	Painted rocks are flat (ignored); real 3-D rocks are 0.2-0.4 m tall obstacles.	The floor ToF jumps over real rocks: height comes from VIO, not ToF.

Risk	Why	What to look for
Battery and RC	Battery sag changes the hover; RC loss triggers the PX4 failsafe.	Fresh battery per block; test the failsafe once with props off.

9. What to send back to Claude

- `course_prior.venue.json` (or the raw numbers), the top-down sketch and stage photos.
- Interface-check outputs: topic list and rates, both camera_info messages, the TF echo, the PX4 parameter dump.
- Walk-through bags (section 6), plus every `flights/<time>_<name>/` folder together with `flights/INDEX.md`.
- Short notes per flight: what happened, battery, any RC takeover, and any other unexpected behaviour.

Claude turns this into: a measured course prior, real calibration in the sim, an offline replay of every detector on real images, and fixes ready before the next test block.

10. Files in the repo

File	Purpose
cartographer/official/SITE_MEASURE.md	Same measurement sheet, in Chinese
cartographer/official/PRIORS.md	Audit of every prior the stack uses (no hard-coded positions)
cartographer/official/OFFICIAL_SIM_SPEC.md	Interfaces and sensor specs the stack expects
cartographer/official/mission/course_prior.example.json	Template for the venue prior
cartographer/official/tools/flight_run.py	Recorded flight launcher (--real on the aircraft)
outputs/flights/INDEX.md	Index of every flight so far (sim and real)